



LOST CHILDREN OF PERU · ARQUITECTURA

Anatomía del Módulo RRHH

Qué contiene el sistema, de qué está hecho cada parte y por qué se decidió así. Para quien tenga que entenderlo entero.

44 280 líneas de código	32 módulos de servidor	128 rutas de API	22 pantallas	29 tablas	90 suites de prueba
---	--	--	------------------------------------	---------------------------------	---

Qué es

Un sistema de recursos humanos para una casa hogar: gestiona al equipo, a los niños acogidos y a sus familias, y registra la asistencia por reconocimiento facial.

Sustituye a un ERP anterior que sigue en producción mientras dura la transición. Cubre seis áreas:

ÁREA	QUÉ RESUELVE
Personal	Fichas, organigrama, documentos con vencimiento, contratos, condiciones y sueldos.
Asistencia	Marcas del terminal biométrico y del celular, en un solo historial.
Permisos	Diez tipos de solicitud, con firma de quien pide y de quien aprueba, y el PDF del formato que la ONG ya usaba en papel.
Planillas	Cálculo mensual y boletas.
Beneficiarios	Los niños: expediente, seguimiento, sesiones de acompañamiento, incidencias, programas e historial escolar.
Responsables	Los tutores y familiares, y su vínculo con cada niño.

Las cuatro capas

Todo corre en un solo proceso y se sirve desde un solo origen. No hay microservicios, ni cola de mensajes, ni caché aparte.

1 · Navegador

La interfaz entera: 22 pantallas y 25 diálogos. Sin framework — el runtime de Claude Design y JavaScript a mano.

HTML

JavaScript

face-api (reconocimiento)

↓ HTTP en el mismo origen, sin CORS ↓

2 · Servidor

Flask. 128 rutas de API más el propio archivo de interfaz. Aquí viven las reglas: permisos, validaciones, el estado del enrolamiento y la generación de PDF.

Python 3.11

Flask

32 módulos

↓ sqlite3, de la librería estándar ↓

3 · Datos

Un archivo SQLite con 29 tablas, y cuatro carpetas de adjuntos al lado: fotos, firmas, documentos y fotos de fichaje.

SQLite

data/rrhh.db

claves foráneas activas

↕ solo cuando hace falta ↕

4 · Fuera

Tres servicios externos, todos opcionales: sin ellos el sistema arranca y funciona, solo pierde esa función.

yunatt (terminal)

Google Sheets (formulario)

OpenStreetMap (lugares)

Un solo origen, y eso simplifica mucho

El servidor sirve la interfaz y la API. No hay CORS que configurar, ni dos despliegues que mantener sincronizados, ni un dominio de front y otro de back. La cookie de sesión vale para todo.

Lenguajes y librerías

Dos lenguajes y tres dependencias. Todo lo demás es librería estándar.

QUÉ	DÓNDE	POR QUÉ
Python 3.11	Servidor	Trae sqlite3, hashlib y http en la librería estándar: casi todo lo que hace falta ya viene dentro.
JavaScript	Navegador	Sin framework. El reconocimiento facial ocurre aquí, no en el servidor.
SQL	Base	Consultas escritas a mano. Sin ORM.
Flask	Servidor	Enrutado y peticiones. Es la única dependencia que toca cada petición.
requests	Servidor	Cliente HTTP hacia yunatt. Su urllib3 permite forzar TLS 1.3, que el handshake con ellos exige en Windows.
gunicorn	Contenedor	Solo en despliegue. En la oficina se usa el servidor de Flask.
face-api 1.7.15	Navegador	Reconocimiento facial. Licencia MIT, alojada en el propio servidor: si no hay internet, sigue funcionando.

Lo que deliberadamente NO se usa

Ni ORM, ni React o Vue, ni Redis, ni Celery, ni servidor de base de datos, ni bundler, ni Node en producción. Cada pieza que no está es una que nadie tiene que actualizar, parchear ni entender dentro de tres años en una ONG sin equipo de sistemas.

El servidor por dentro

32 módulos, 15 522 líneas. Dos concentran la mitad, y es a propósito.

MÓDULO	LÍNEAS	QUÉ HACE
app.py	3 219	Las 128 rutas. Cada una valida, comprueba permisos y delega: la lógica no vive aquí.
db.py	3 137	El esquema y todo el SQL. Ningún otro módulo escribe SQL.
yunatt_client.py	888	Todo lo que habla con el terminal, aislado en un solo sitio.
enrolamiento.py	623	La máquina de estados de la captura biométrica.
reportes.py	595	Un PDF por módulo, con los filtros que se estén viendo.
config.py	547	Variables, catálogo de módulos, niveles de permiso y el rango reservado.
auth.py	410	Identidad, sesiones y permisos.
pdf.py	311	Genera PDF sin instalar nada: escribe el formato a mano.
sincronizador.py	110	Trae las marcas del terminal cada 45 segundos, en un hilo de fondo.

El resto se reparte entre reglas de negocio (`solicitudes` , `planillas` , `personas`), adjuntos (`fotos` , `firmas` , `archivos`), integraciones (`google_hoja` , `formulario` , `lugares`), utilidades de arranque (`servidor` , `wsgi` , `crear_director`) y **cinco migraciones** con nombre propio, que se conservan porque documentan cómo evolucionó el esquema.

La regla que sostiene todo lo demás

El SQL vive solo en `db.py` . Ninguna ruta, ninguna regla de negocio y ninguna integración escribe una consulta. Cuando hay que cambiar cómo se guarda algo, se cambia en un archivo — y las decisiones delicadas, como qué significa exactamente «enrolado», tienen una sola definición que nadie puede contradecir por descuido.

La interfaz

22 pantallas, 25 diálogos, 8 417 líneas de lógica. Y un detalle que hay que saber antes de tocar nada.

El archivo que sirve el navegador es generado

ERP RRHH - Lost Children Peru.dc.html se reconstruye en cada arranque a partir de interfaz/. Editarlo a mano es perder el trabajo en el siguiente reinicio.

CARPETA	ARCHIVOS	CONTENIDO
interfaz/pantallas	22	Una pieza por pantalla: dashboard, directorio, asistencia, permisos, planillas, beneficiarios...
interfaz/dialogos	25	Los cuadros: ficha, cámara, firma, permiso, borrados, documento...
interfaz/logica	17	El JavaScript, un archivo por área.
interfaz/base.html	1	El armazón: barra lateral, cabecera, pie.

Cómo se actualiza sola

Cada pantalla pregunta a /api/novedades cada 5 segundos. Esa ruta no devuelve datos: devuelve un sello que cambia cuando cambia algo. Si el sello es el mismo, la pantalla no hace nada; si cambió, recarga solo lo que está mirando. Con la pestaña de fondo ni se pregunta — en un teléfono eso es batería y datos regalados.

Se eligió sondeo en vez de websockets por una razón concreta: el servidor corre con **un solo trabajador y ocho hilos**, y una conexión permanente por pestaña abierta consumiría un hilo cada una. Con ocho pestañas el sistema dejaría de atender nada, ni siquiera marcar.

Los datos

29 tablas y una vista, en un solo archivo.

GRUPO	TABLAS
Personas	personal , beneficiarios , responsables , responsable_beneficiario
Asistencia	identidades , marcas , rostros_web , consentimientos
Expediente	documentos , condiciones_laborales , formacion , experiencia , boletas
Niños	seguimiento , sesiones_acompanamiento , incidencias , programas_beneficiario , historial_educativo
Permisos	solicitudes
Acceso	usuarios , roles , permisos_rol , sesiones_usuario , intentos_login , accesos , invitaciones
Sistema	parametros , respuestas_formulario , personas_migrada

La vista que evita una contradicción

`v_identidades` existe para que «enrolado» tenga **una sola definición**. No es «tiene fila» —la fila se crea al pedir el enrolamiento, antes de mandar nada al terminal— ni es `estado='enrolado'`, que solo cuenta cómo fue la conversación con el equipo. Enrolado es **lo que el terminal confirmó que guardó**: tiene rostro o tiene huella. Sin eso la persona no puede fichar, y decir lo contrario sería informar en falso sobre alguien real.

Un respaldo del .db sin sus carpetas está incompleto

La base guarda el **nombre** de cada archivo, no el archivo. `data/fotos` , `data/firmas` , `data/archivos` y `data/marcas` viajan con ella o la restauración deja las fichas apuntando a documentos que no existen.

Quién ve qué

18 módulos × 3 niveles, por rol. No hay permisos por persona: se administran roles.

NIVEL	QUÉ PERMITE
ninguno	Ni siquiera aparece en el menú. No es que esté deshabilitado: no existe para esa persona.
vista	Puede mirar. Ninguna escritura pasa.
edición	Puede cambiar.

Los 18 módulos se agrupan en seis ámbitos: **General** (dashboard, reportes), **Personal** (directorio, organigrama, documentos, contratos, condiciones), **Operación** (asistencia, permisos, planillas), **Beneficiarios** (fichas, responsables, sesiones, incidencias), **Otros** (capacitaciones, evaluaciones) y **Sistema** (configuración, usuarios).

Lo que protege de verdad

- La comprobación está en el `server.js`, en un decorador por ruta. Ocultar un botón no protege nada: quien `server.js` ca la dirección la llama igual.

- **Las contraseñas**

se guardan como hash PBKDF2-SHA256 con 240000 iteraciones.

- **Los tokens de sesión**

tampoco
se
guardan:
en
la
tabla
queda
su
huella
SHA-
256,
así
que
quien
lea
la
base
no
puede
suplantar
a
nadie.

■ **Toda escritura**

exige
el
token
CSRF.

■ **La sesión caduca**

a
los
45
minutos
sin
uso.

■ **Quien aprueba queda anotado**

:
la
firma

de
un
permiso
sale
de
la
sesión
de
quien
lo
aprobó,
nunca
de
lo
que
venga
en
la
petición.

Los dos caminos de fichar

Terminal y celular. Dos tecnologías distintas, un solo historial.

	TERMINAL (TIMMY)	CELULAR
Dónde compara	Dentro del equipo	En el navegador de la persona
Qué se guarda	La plantilla vive en el terminal y en yunatt	128 números aquí. Nunca la foto de referencia
Necesita internet	Sí, pasa por yunatt	No: los modelos se sirven desde el propio servidor
Canal en la tabla	terminal	web
Además guarda	La foto que tomó al enrolar, que va a la ficha	Foto del fichaje y el nombre del lugar

Los dos escriben en `marcas` , con una columna `canal` que dice de dónde vino cada una. Un solo historial porque la pregunta que importa —¿vino hoy?— no cambia según cómo fichara; pero se puede distinguir cuando hace falta.

El reconocimiento del celular, en detalle

■ Todo ocurre en el navegador.

La cara no se manda a ningún servidor para compararla.

■ Se guardan 128 números

, no

la
fotografía.
De
ese
vector
no
se
reconstruye
una
cara.

■ **Umbral 0,6**

,
medido:
la
misma
cara
consigo
misma
da
entre
0,03
y
0,14;
caras
distintas,
entre
0,64
y
0,89.

■ **El nombre del lugar**

se
resuelve
una
vez,
al
guardar,
nunca
al
pintar

una
pantalla:
hacerlo
al
mirar
mandaría
la
ubicación
de
todo
el
equipo
a
un
tercero
cada
vez
que
alguien
abre
el
registro.

Cambiar de modelo obliga a re-enrolar a todo el mundo

Los 128 números solo tienen sentido dentro del modelo que los calculó.

Lo que habla con fuera

Tres servicios, y ninguno es imprescindible para que el sistema arranque.

SERVICIO	PARA QUÉ	SI FALLA
yunatt	Enrolar en el Timmy y traer sus marcas.	Todo lo demás sigue. Enrolar queda en cola y se manda solo al volver; las marcas del celular no se enteran.
Google Sheets	Traer las respuestas del formulario de tutores.	Se dejan de traer respuestas nuevas. Las ya traídas están en la base.
OpenStreetMap	Poner nombre a las coordenadas de un fichaje.	La marca entra igual, sin nombre de lugar. Se apaga con <code>LUGARES_ACTIVO=0</code> .

Ese aislamiento se ganó a base de disgustos

El 1 de setiembre de 2026 la plataforma de yunatt se cayó entera. El sistema siguió funcionando: fichas, asistencia por celular, permisos y reportes, todo normal. Solo se detuvo lo que necesitaba su servidor.

Cómo se comprueba

90 suites, 15 911 líneas. Casi tanto código de prueba como de servidor, y no es casualidad.

Dos clases: unas hablan con la API y comprueban reglas; otras abren un **navegador de verdad**, pulsan botones y leen lo que aparece en pantalla. Las segundas son las que atrapan lo que un test de API no ve —un botón que no llama a nada, un diálogo que no cierra, un dato que llega pero no se pinta.

```
# Todas
cd pruebas
py correr_todo.py

# Una sola
py correr_una.py prueba_marcar_foto

# Antes de dar por bueno cualquier cambio de interfaz
py verifica_ids.py
```

Nunca las dos cosas a la vez

La corrida completa y una suite suelta usan el mismo puerto 7802. Lanzar una mientras corre la otra produce decenas de fallos falsos que no tienen nada que ver con el sistema. Lo mismo vale para reconstruir la interfaz a mitad de una corrida.

Cada corrida trabaja sobre una **copia** de la base y la borra al terminar. Hay una suite dedicada que lo comprueba matando el proceso en seco y verificando que la base real quedó igual.

Por qué así

Las decisiones que más condicionan el sistema, y qué haría falta para cambiarlas.

DECISIÓN	MOTIVO	PARA CAMBIARLA
SQLite y no un servidor de base	Un archivo. Nada que instalar, arrancar ni administrar en una ONG sin equipo de sistemas.	Migrar el esquema y sustituir <code>db.py</code> . El resto del código no lo notaría.
Un solo trabajador	El seguimiento del enrolamiento vive en la memoria del proceso.	Mover ese estado a la base. Entonces se podría escalar y usar websockets.
Sin framework en la interfaz	Sin bundler, sin Node en producción, sin cadena de dependencias que envejezca.	Reescribir la interfaz entera.
Sondeo cada 5 s	Consecuencia del trabajador único.	Va junto con lo anterior.
Números ≥ 9000	Barrera para no pisar al ERP anterior cuando compartían cuenta.	Se puede levantar cuando el ERP viejo se apague.
Todo el SQL en un módulo	Una sola definición para las cosas delicadas.	No conviene cambiarla.

El hilo común

Cada decisión está tomada pensando en quién mantiene esto dentro de tres años, no en quién lo escribe hoy. Por eso hay pocas piezas, por eso el código explica *por qué* y no solo *qué*, y por eso las pruebas ocupan tanto como el sistema.